

MAVLink 协议调研与开发环境搭建报告

环境配置、SITL 仿真与 QGroundControl 连接验证

实习周报成果

2026 年 7 月 1 日

目录

1	MAVLink 协议调研	3
1.1	MAVLink 概述	3
1.2	与其他通信方案的对比	3
1.3	系统架构与角色划分	4
1.4	技术选型分析	4
1.4.1	MAVLink 与物理层的适配	4
1.4.2	选型理由：Python + ArduPilot SITL + QGC	6
2	开发环境配置文档	6
2.1	总体环境说明	6
2.2	Windows 侧环境配置流程	6
2.2.1	安装 Git 并修复 SSL 问题	6
2.2.2	安装 QGroundControl	6
2.2.3	安装 Visual Studio Build Tools (MSVC)	6
2.2.4	安装与配置 Qt 5.15.2	6
2.3	Ubuntu 虚拟机侧：ArduPilot SITL 配置	7
2.3.1	创建虚拟机并安装 Ubuntu	7
2.3.2	克隆 ArduPilot 源码	7
2.3.3	安装 Python 与系统依赖	7
2.3.4	首次启动 SITL	7
2.4	环境配置中的主要难点与解决方案	8
3	连接与验证	8
3.1	网络连接配置	8

3.2	QGC 连接 SITL 验证	9
3.3	MAVLink Inspector 消息观察	9
3.4	操作与指令消息联动验证	9
3.5	验证截图	10
4	总结与后续工作	10

1 MAVLink 协议调研

1.1 MAVLink 概述

MAVLink (Micro Air Vehicle Link) 是一种轻量级、开源的无人机通信协议，最初由苏黎世联邦理工学院 (ETH Zurich) 的 Lorenz Meier 等人于 2009 年前后提出，旨在为微型飞行器提供统一的机载消息格式与地面站交互能力。经过多年发展，MAVLink 已成为开源飞控生态 (ArduPilot、PX4 等) 与地面站 (QGroundControl、Mission Planner 等) 之间通信的事实标准。

设计目标主要包括：

- **轻量：**单条消息头部仅 10 字节 (MAVLink 2)，适合带宽受限的无线链路；
- **开源：**消息定义以 XML 描述，可自动生成多语言编解码库；
- **通用：**覆盖心跳、姿态、GPS、任务、参数、命令等数百种标准消息类型；
- **可扩展：**支持厂商自定义消息及 MAVLink 2 的签名认证机制。

当前行业地位：在开源无人机领域，MAVLink 2 几乎是飞控与地面站之间的默认应用层协议；商用领域亦广泛采用或兼容该协议栈。

1.2 与其他通信方案的对比

表 1: MAVLink 与 DDS、ROS 通信层对比

维度	MAVLink	DDS	ROS/ROS2 通信
定位	无人机专用应用层消息协议	通用分布式中间件	机器人框架内置通信层
消息模型	预定义二进制消息 ID + 载荷	Topic/Service, IDL 定义	Topic/Service/Action
典型链路	串口、UDP、TCP	以太网、共享内存等	共享内存、UDP、DDS (ROS2)
资源占用	极低，适合嵌入式	相对较高	中等，依赖节点图
适用场景	机载飞控 ↔ 地面站/ companion	多节点分布式系统、 自动驾驶	机载计算、仿真、算 法验证

小结：MAVLink 面向“飞控—地面站”这一垂直场景做了高度优化；DDS 更适合复杂分布式系统；ROS 通信层更适合机载算法与仿真生态。本实习以 MAVLink 为主线，因其与 QGC、ArduPilot SITL 的直接兼容性最好。

1.3 系统架构与角色划分

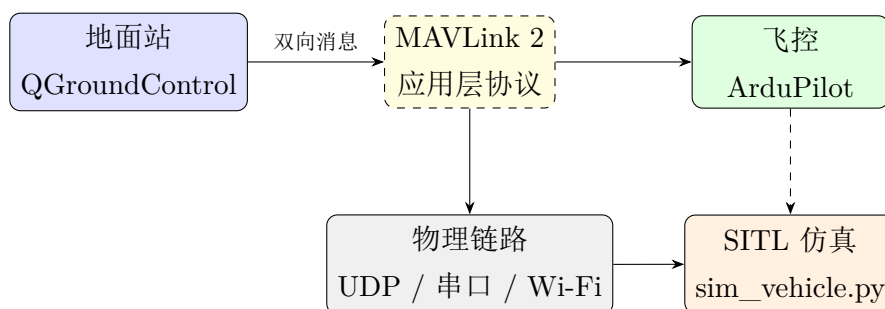


图 1: 地面站—协议—飞控（SITL）关系示意

各组件角色如下：

- **QGroundControl (QGC)**: 地面控制站，负责任务规划、参数配置、飞行监控及 MAVLink 消息可视化；
- **ArduPilot**: 开源飞控固件；在 SITL 模式下于主机/虚拟机中模拟真实飞控行为；
- **MAVLink**: 定义消息格式与语义，与底层传输无关；
- **物理链路**: 本实验采用 UDP（默认端口 14550）；实际飞行可使用数传电台、Wi-Fi、4G 等；
- **Python / pymavlink**: 用于脚本级 MAVLink 交互，适合自动化测试与二次开发。

1.4 技术选型分析

1.4.1 MAVLink 与物理层的适配

MAVLink 作为应用层协议，通过 `mavlink_connection` 等接口适配多种传输方式：

- **串口 (Serial)**: 典型用于飞控直连或数传电台，波特率常见 57600/115200；
- **UDP**: SITL 仿真与地面站本地通信的首选，延迟低、配置简单；
- **TCP**: 适用于可靠连接场景, SITL 内部亦使用 TCP 5760 连接仿真核心与 MAVProxy。

表 2: 本实验技术栈选型理由

组件	选型理由
Python + py-mavlink	语法简洁、库成熟、便于快速验证消息收发与后续脚本开发
ArduPilot SITL	无需真实飞机即可复现完整飞控逻辑，与 QGC 生态无缝衔接
QGroundControl	开源、支持 MAVLink 2、内置 MAVLink Inspector，便于教学与调试
Ubuntu 虚拟机	ArduPilot 官方工具链面向 Linux，在 Windows 宿主上通过 VM 运行 SITL 是常见方案

表 3: 软硬件环境一览

项目	配置
宿主操作系统	Windows 10（版本 10.0.22000）
虚拟机	VMware Workstation + Ubuntu（桌面版）
宿主 Python	Python 3.10.7
地面站	QGroundControl 4.2（支持 MAVLink 2）
飞控仿真	ArduPilot SITL（ArduCopter, Copter-4.4 分支）
Windows 编译工具链（可选）	Visual Studio Build Tools（MSVC v14.29），路径 D:\Micros
Qt 环境（可选，源码编译 QGC 用）	Qt 5.15.2 MSVC2019 64-bit, Qt Creator, CMake 3.29.3, 路
版本管理	Git 2.47.0

1.4.2 选型理由: Python + ArduPilot SITL + QGC

2 开发环境配置文档

2.1 总体环境说明

本报告实验路径采用 Windows 宿主运行 QGC + Ubuntu 虚拟机运行 ArduPilot SITL 的分离架构; Qt/MSVC 环境作为从源码构建 QGC 的预备能力一并记录。

2.2 Windows 侧环境配置流程

2.2.1 安装 Git 并修复 SSL 问题

安装 Git for Windows 后, 克隆 GitHub 仓库时出现:

```
1 fatal: unable to access 'https://github.com/...':  
2 SSL certificate problem: unable to get local issuer certificate
```

解决方案: 将 Git SSL 后端切换为 Windows 原生证书库:

```
1 git config --global http.sslBackend schannel
```

2.2.2 安装 QGroundControl

从 QGC 4.2 发布页下载 QGroundControl-installer.exe 并安装。QGC 4.2 默认支持 MAVLink 2, 无需额外配置协议版本。

2.2.3 安装 Visual Studio Build Tools (MSVC)

为后续可选的 QGC 源码编译, 安装 Visual Studio Build Tools, 勾选“使用 C++ 的桌面开发”, 包含 MSVC v142 与 Windows 10/11 SDK。实际安装路径为 D:\Microsoft Visual Studio (非默认 C 盘路径), 导致初期检测脚本未识别, 需手动在 Qt Creator 中配置编译器。

2.2.4 安装与配置 Qt 5.15.2

通过 Qt 维护工具 (MaintenanceTool) 在已有 Qt 6.8.0 环境中追加安装 Qt 5.15.2 MSVC2019 64-bit, 路径为 D:\Qt6\5.15.2\msvc2019_64。

Qt Creator Kit 配置难点:

1. 新建 Kit 后 Compiler 显示 <No compiler>, 需手动添加 MSVC;
2. Initialization 须指向 vcvarsall.bat 并选择参数 amd64, 不可使用 x86;

3. ABI 须为 x86-windows-msvc2019-pe-64bit, 与 Qt 5.15.2 MSVC2019 套件匹配;
4. C 与 C++ 编译器需分别添加并绑定到 Kit。

2.3 Ubuntu 虚拟机侧: ArduPilot SITL 配置

2.3.1 创建虚拟机并安装 Ubuntu

在 VMware 中创建 Ubuntu 虚拟机, 分配不少于 4GB 内存与 40GB 磁盘。ArduPilot 官方工具链主要面向 Linux, 故 SITL 运行于虚拟机内。

2.3.2 克隆 ArduPilot 源码

```
1 cd ~
2 git clone https://github.com/ArduPilot/ardupilot.git
3 cd ardupilot
4 git checkout Copter-4.4
5 git submodule update --init --recursive
```

源码存放于 / 桌面/ArduPilot/ardupilot。必须使用 `--recursive` 或 `submodule update`, 否则依赖库缺失将导致编译失败。

2.3.3 安装 Python 与系统依赖

```
1 cd ~/ardupilot
2 Tools/environment_install/install-prereqs-ubuntu.sh -y
```

安装完成后需关闭并重新打开终端, 使命令行环境变量生效。此后可直接使用 `sim_vehicle.py`, 或显式调用:

```
1 python3 ../Tools/autotest/sim_vehicle.py -v ArduCopter --console --
    map
```

2.3.4 首次启动 SITL

首次运行会自动编译 ArduCopter SITL 目标, 耗时约 2-30 分钟。成功标志包括:

- build finished successfully
- Detected vehicle 1:1 on link 0
- Received 1339 parameters
- 控制台提示符为 STABILIZE>

2.4 环境配置中的主要难点与解决方案

问题现象	原因分析	解决方法
Git 克隆 SSL 失败	OpenSSL 证书链与系统不一致	<code>git config --global http.sslBackend schannel</code>
Qt 在线安装器验证失败	账号/网络无法访问 Qt 服务器	使用离线安装包或国内镜像；网页重置密码后重试
Qt Kit 黄色警告	MSVC 未绑定或 ABI/架构错误	手动添加 <code>vcvarsall.bat + amd64</code> ，绑定 Qt 5.15.2
<code>sim_vehicle.py</code> : 未找到命令	未执行 <code>install-prereqs</code> 或 <code>PATH</code> 未生效	运行官方安装脚本并重启终端
UDP ports must be specified as host:port	<code>--out</code> 参数缺少端口号	使用 <code>--out=udp:IP:14550</code> 格式
使用公网 IP（如 26.x.x.x）无法连接	虚拟机路由不可达该地址	改用桥接模式 + Windows 局域网 IP
ping 宿主机失败	虚拟机为 NAT 模式	改为桥接模式并重启虚拟机
SITL 正常但 QGC 仍 Disconnected	MAVLink 未发往 Windows；或端口冲突	添加 <code>--out=udp:WindowsIP:14550</code> ；关闭 QGC 中 NMEA 对 14550 端口的占用；放行防火墙 UDP 14550

表 4: 环境搭建主要问题与解决方案

3 连接与验证

3.1 网络连接配置

最终将 VMware 网络适配器设置为桥接模式，使 Ubuntu 虚拟机与 Windows 宿主处于同一局域网。在 Windows 上通过 `ipconfig` 获取 IPv4 地址（形如 192.168.x.x），在虚拟机中启动 SITL 时显式指定 MAVLink 输出：

```
1 cd ~/桌面/ArduPilot/ardupilot/ArduCopter
```



```

2 python3 ../Tools/autotest/sim_vehicle.py -v ArduCopter --console --
  map \
3   --out=udp:<Windows 主机IP>:14550

```

QGC 侧在“应用设置”中启用 UDP 自动连接；需确保 NMEA GPS 流不使用 14550 端口，以免与 MAVLink 默认监听端口冲突。Windows 防火墙需允许入站 UDP 14550。

3.2 QGC 连接 SITL 验证

连接成功后，QGC 顶部状态由 `Disconnected` 变为 `Connected`，地图显示仿真飞机位置，遥测数据（高度、速度等）开始刷新。

3.3 MAVLink Inspector 消息观察

在 QGC 中选择 **Analyze** → **MAVLink Inspector**，对以下 MAVLink 2 消息进行观察与记录：

表 5: 关键 MAVLink 2 消息说明

消息名称	观察要点
HEARTBEAT	心跳包，约 1 Hz 周期刷新；含飞行器类型、飞行模式、系统状态
GPS_RAW_INT	原始 GNSS 数据：纬度、经度、高度及定位精度相关字段
SYS_STATUS	系统状态：传感器健康、电池电压、负载等
COMMAND_LONG	地面站下发的长命令（解锁、起飞、模式切换等）

3.4 操作与指令消息联动验证

在 QGC 飞行界面依次执行解锁、起飞与飞行模式切换操作，并在 MAVLink Inspector 中同步观察对应消息。需要说明的是，Inspector 默认主要展示从飞控接收的消息；地面站下发的 `COMMAND_LONG` 不一定出现在列表中，但飞控返回的 `COMMAND_ACK`（命令确认）可在操作瞬间观察到，同样能够证明指令已成功下发并被飞控接受。

1. **解锁 (Arm)**：Inspector 中出现 `COMMAND_ACK`，`command` 字段对应解锁相关命令，`result=0` 表示命令被接受；
2. **起飞 (Takeoff)**：同样可观察到 `COMMAND_ACK` 消息刷新；

3. 切换飞行模式: COMMAND_ACK 中 `command=176` (MAV_CMD_DO_SET_MODE), `result=0` 表示模式切换成功。

上述操作与消息观察均已完成,证明地面站、MAVLink 协议栈与 SITL 仿真飞控之间通信正常。验证截图见下文图 2-图 5。

3.5 验证截图

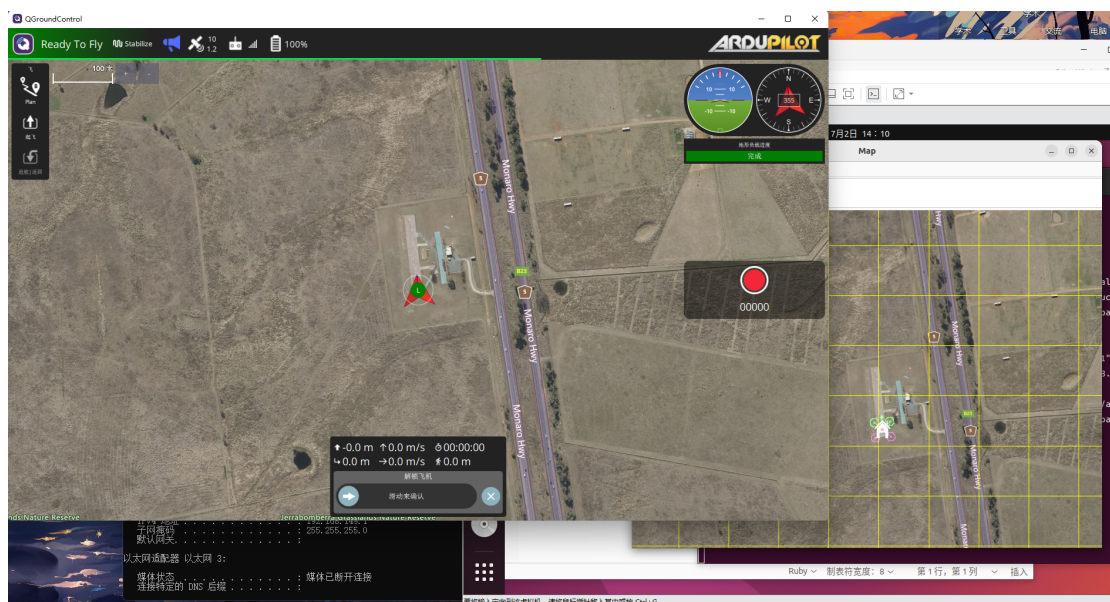


图 2: QGC 与 SITL 连接成功界面

4 总结与后续工作

本周完成了 MAVLink 协议调研、Windows/Linux 混合开发环境搭建、ArduPilot SITL 仿真启动以及 QGroundControl 与 SITL 的 MAVLink 2 连接验证。环境搭建过程中,Qt Kit 配置、虚拟机网络桥接与 MAVLink UDP 端口转发是主要难点,均已通过查阅文档与逐步排查解决。

后续计划(对应实习任务安排):

- 使用 Python 与 pymavlink 编写 `basic_communication.py` 等基础交互脚本;
- 实现参数读写、任务上传、命令控制等高级功能;
- 启用并验证 MAVLink 2 安全签名;
- 完成地理围栏监测与自动悬停/返航脚本开发。

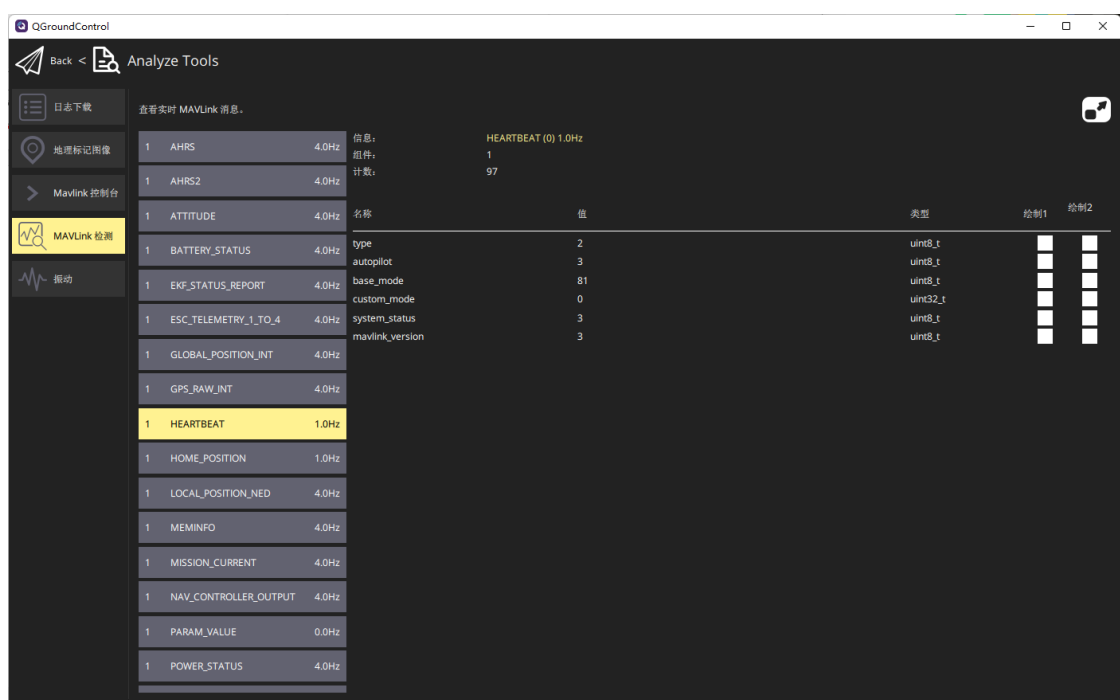


图 3: MAVLink Inspector 关键消息观察 (HEARTBEAT、GPS_RAW_INT、SYS_STATUS 等)

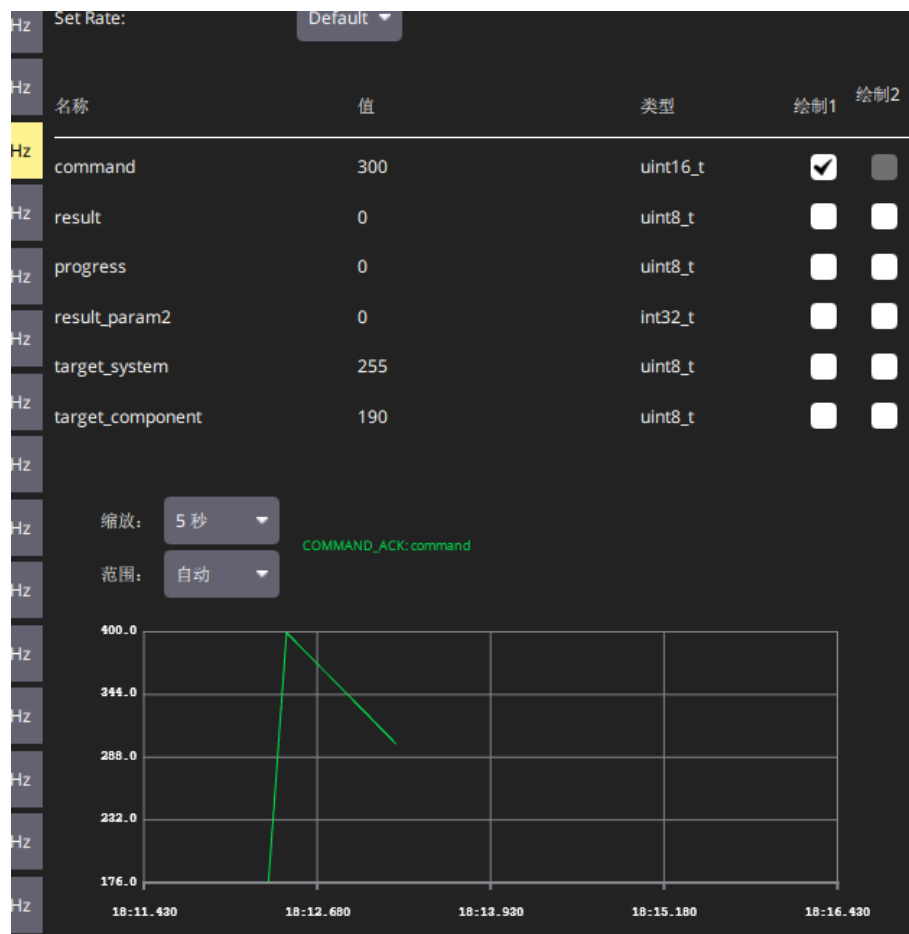


图 4: 解锁/起飞操作触发的 COMMAND_ACK 消息 (command 字段变化, result=0 表示命令被接受)



图 5: 飞行模式切换时的 COMMAND_ACK 消息 (command=176, MAV_CMD_DO_SET_MODE)